

七星微赛题初赛设计说明书

参赛作品：YH_rv_cpu

RISC-V 处理器与最小 SoC 设计说明

版本：v0.4 日期：2026-04-09

本文档说明 YH_rv_cpu 的设计目标、总体架构、关键微架构、验证结果以及 FPGA 原型实现情况，并对仍待实体板验证的事项作出说明。

摘要

YH_rv_cpu 是一套面向七星微赛题的 RISC-V 处理器方案。设计采用五级顺序流水结构，围绕 RV32I + Zicsr 构建同步指令存储、数据存储、UART、定时器和 DONE 标志构成的最小 SoC，形成了从功能验证、性能评估到 FPGA 原型实现的完整工程链路。本文对该设计的目标、总体架构、关键机制、验证结果和原型实现情况进行说明。

截至 2026 年 4 月，本设计已完成基础回归、扩展用户态验证、CoreMark short 与 strict 长跑测试，以及 Vivado impl50 综合实现。关键结果包括：CoreMark short 0.912472 CoreMark/MHz、CoreMark strict 0.912465 CoreMark/MHz、rv32 baseline 33/33、rv64 baseline 21/21、rv32 full-ui 42/42、rv64 full-ui 54/54，以及 2556 LUT / 2170 FF / 4 BRAM / 0 DSP 的实现结果。

本设计的特点在于结构边界清晰、验证证据完整、工程结果可复核，并已具备继续开展 FPGA 实体板验证的基础。

关键词

RISC-V；五级流水；工程化验证；CoreMark；FPGA 原型

目录

摘要	1
关键词	1
1 设计目标与方案定位	3
1.1 题目要求分析	3
1.2 设计目标	3
2 总体方案与微架构设计	4
2.1 系统总体架构	4
2.2 CPU 五级流水组织	5
2.3 数据相关处理	6
2.4 控制相关与 redirect 机制	6
2.5 CSR、异常与中断处理	6
2.6 SoC 存储与 MMIO 组织	7
2.7 架构取舍	7

3	验证方案与实验结果	7
3.1	验证矩阵	7
3.2	功能验证结果	8
3.3	CoreMark 性能结果	8
3.4	FPGA 实现结果	9
3.5	验证结论	9
4	FPGA 原型实现情况	10
4.1	原型目标与实现路径	10
4.2	已完成内容	10
4.3	关键结果	11
4.4	待验证事项	11
4.5	小结	11
5	结论	11
5.1	综合结论	11
5.2	待继续验证事项	12
5.3	结语	12
A	关键结果汇总	13

1 设计目标与方案定位

1.1 题目要求分析

结合官方赛题描述与已公开答疑信息，本项目对题目的理解可以概括为以下几点：

- 设计需要在功能正确、性能表现、资源开销和工程可实现性之间取得平衡；
- 处理器方案应具备清晰的架构边界，能够支撑技术说明、验证和后续扩展；
- 程序存储与数据存储方式可根据实现需要选取，FPGA 原型实现结果可作为阶段性支撑证据；
- 在初赛阶段，建立一套能够稳定复现、便于验证且可继续演进的工程基线具有实际意义。

考虑到上述要求，本设计采用五级顺序流水结构，以 RV32I + Zicsr 为核心指令子集，围绕最小 SoC 和 FPGA 原型实现建立完整工程链路。这样的选择有利于在结构复杂度、验证成本和工程可复现性之间取得平衡，并为后续性能优化、扩展指令支持和实体板验证保留清晰接口。

1.2 设计目标

本项目在初赛阶段设置了四项核心目标：

1. **架构清晰**：构建层次分明、职责明确的 CPU 与 SoC 结构，便于说明微架构原理；
2. **验证充分**：建立从 smoke、baseline、扩展 ISA 覆盖到 CoreMark 和实现结果的验证矩阵；
3. **工程可复现**：使关键脚本、文档、RTL、测试平台和结果摘要保持一致；
4. **原型可延伸**：在保持实现约束可控的前提下，为 FPGA 原型和后续性能优化保留清晰路径。

表 1: 初赛阶段实现内容概览

目标项	实现状态	说明
五级流水实现	已完成	CPU、SoC、取指存储、数据存储和 MMIO 路径已形成稳定实现
功能验证	已完成	Smoke、rv32 baseline 33/33、rv64 baseline 21/21 已完成
扩展覆盖	已完成	rv32 full-ui 42/42、rv64 full-ui 54/54 已完成，用于补充展示设计健壮性
性能评估	已完成	CoreMark short 与 strict >=10s 路径均已获得稳定结果
FPGA pre-board	已完成	impl50、资源与时序报告、bring-up 路径已整理完毕
实体板验证	待完成	受实体板卡与最终板级约束条件限制，留作后续阶段工作

本文的主要说明对象是 RV32I + Zicsr 处理器及其最小 SoC 实现；同时列出扩展 full-ui 和 RV64 相关验证结果，用于说明该设计在更广测试范围下的稳定性与可复用性。

2 总体方案与微架构设计

2.1 系统总体架构

YH_rv_cpu 采用“CPU 核 + SoC 封装 + FPGA 原型顶层”的层次化组织方式。CPU 内核负责指令执行、控制流处理和异常响应；SoC 层负责连接同步指令存储、数据存储和最小 MMIO 外设；FPGA 顶层负责将工程映射到 Nexys A7-100T 的原型实现路径。

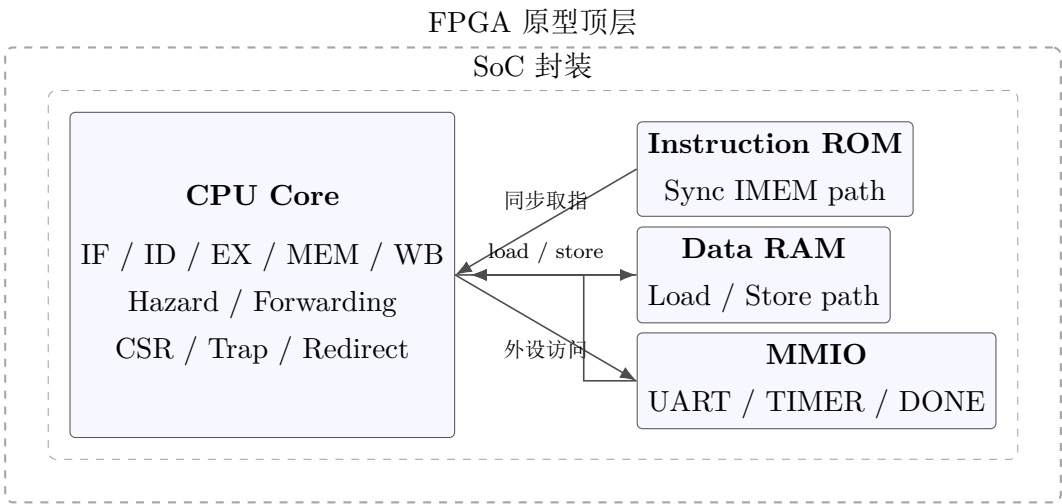


图 1: 系统总体架构示意图

表 2: 核心模块职责

模块	作用
YH_rv_cpu	CPU 顶层，组织 IF/ID/EX/MEM/WB 五级流水、前递、redirect、CSR 与 trap
YH_rv_cpu_hazard_unit	负责 load-use hazard 检测和前递选择，是停顿控制的关键模块
YH_rv_cpu_soc	SoC 封装层，连接同步指令存储、数据存储与最小 MMIO 外设
YH_rv_sync_imem_rom	服务同步取指路径，使 CPU 与 FPGA 原型实现保持一致
YH_rv_dmem_ram	负责 load/store 数据访问，并与 MMIO 共同组成访问路径
YH_rv_cpu_fpga_top	FPGA 原型顶层与综合参数入口，用于承接 impl50 等实现流程

2.2 CPU 五级流水组织

处理器采用经典五级流水：

1. IF：同步取指、PC 更新与 redirect 接收；
2. ID：译码、寄存器读、立即数生成和控制信号准备；
3. EX：ALU 运算、分支跳转判断、CSR 访问以及 trap/mret 决策；
4. MEM：load/store 与 MMIO 访问；
5. WB：将 ALU、访存或 CSR 结果回写寄存器堆。

这种结构的优势在于边界清楚、验证成本可控，并且足以支撑 CoreMark、riscv-tests 和最小 SoC 软件运行。初赛版本没有引入 cache、多发射或乱序机制，而是优先保证结构可说明、行为可验证、结果可复现。

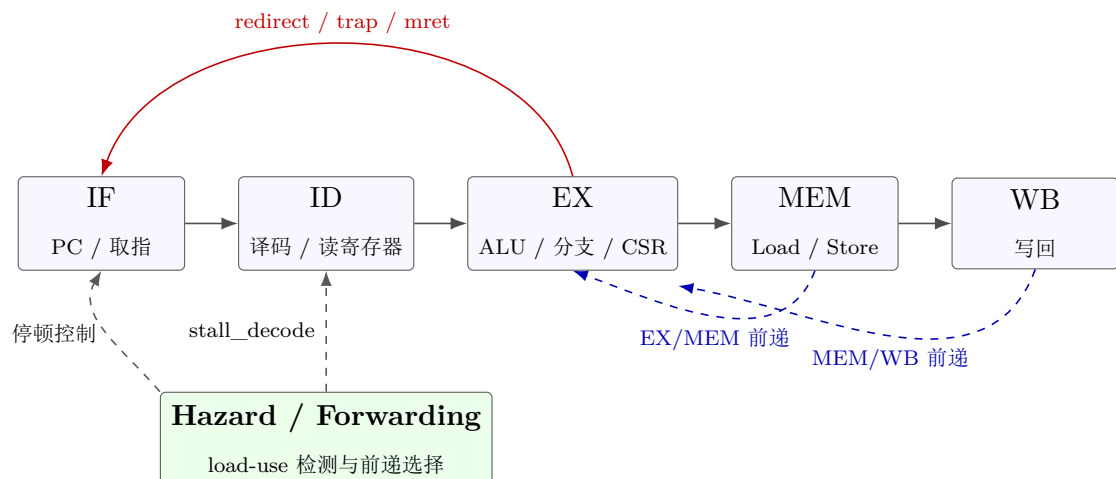


图 2: 五级流水与关键控制通路示意图

2.3 数据相关处理

为了在顺序流水中兼顾正确性和吞吐，项目实现了明确的数据相关处理策略：

- EX/MEM、MEM/WB 两级前递，降低普通 ALU 相关导致的停顿；
- 针对 load-use 场景设置专门检测逻辑，当结果尚未可用且后继指令立即消费时暂停译码级；
- 译码级停顿条件采用 `stall_decode = load_use_hazard` 策略，使停顿范围尽量收敛在必要场景内。

这一选择的直接效果是：对可前递场景尽量放行，对不可前递的 load-use 场景执行最小必要停顿，在结构复杂度、稳定性和性能之间取得平衡。

2.4 控制相关与 redirect 机制

控制流处理主要在 EX 级完成决策：

- branch、jump 和 jalr 在 EX 级形成 redirect；
- trap、mret 与普通跳转共享控制流重定向框架；
- 当 `ex_fetch_redirect_valid` 有效时，IF 级更新 PC 并清空错误路径气泡。

这种组织方式能够用统一框架处理普通跳转、异常进入和异常返回，既保持控制路径简洁，也便于验证 trap 与正常执行之间的切换关系。

2.5 CSR、异常与中断处理

本设计实现的控制状态寄存器与异常处理，重点覆盖比赛阶段确有需求的机器态功能：

- CSR 指令：`csrrw/csrrs/csrrc` 及其立即数变体；
- 关键机器态 CSR：`mstatus`、`mie`、`mtvec`、`mscratch`、`mepc`、`mcause`、`mip`；
- 同步异常：非法指令、CSR 非法访问、load misaligned 等；
- 控制流相关：`ecall`、`ebreak`、`mret`；
- 机器定时器中断：通过 `timer` 外设触发并进入 trap 流程。

这一范围能够满足现阶段软件运行和验证需要，也有利于将实现重点集中在基础执行链路、异常响应和工程完整性上。

2.6 SoC 存储与 MMIO 组织

SoC 采用“ROM + RAM + MMIO”的最小闭环结构。其中同步 ROM 负责取指, RAM 负责数据访问, MMIO 外设负责提供对外可观测接口和计时能力。关键地址组织如下:

表 3: SoC 关键地址映射

资源	地址/基址	作用
ROM	0x0000_0000	程序存储基址
RAM	0x0000_4000	数据存储基址
UART TX	0x1000_0000	串口发送寄存器
DONE	0x1000_0004	程序结束标志寄存器
TIMER VALUE LO	0x1000_0008	计时值低 32 位
TIMER VALUE HI	0x1000_000C	计时值高 32 位
TIMER CMP LO	0x1000_0010	比较寄存器低 32 位
TIMER CMP HI	0x1000_0014	比较寄存器高 32 位
TIMER CTRL	0x1000_0018	定时器控制与中断使能

这种组织方式具有三方面优势: 一是能够直接支撑仿真、CoreMark 和 `riscv-tests`; 二是向 FPGA 原型迁移成本较低; 三是 UART、timer 和 DONE 为后续 bring-up 提供了最小但有效的观测接口。

2.7 架构取舍

本设计优先采用边界清晰、易于验证和便于复现的结构, 重点保留已经通过验证矩阵的机制与参数, 暂不引入 cache、多发射、乱序等更高风险的复杂结构。这样的取舍使得文中的关键结论能够被实验结果直接支撑, 同时也为后续性能优化和实体板验证保留了清晰起点。

3 验证方案与实验结果

3.1 验证矩阵

本设计围绕“功能正确、性能可信、实现可落地”建立了分层验证矩阵:

- Smoke 回归: 快速确认脚本、编译链路和最小软件路径可运行;
- Baseline `riscv-tests`: 验证目标 ISA 集合的基本正确性;
- 扩展 `full-ui`: 展示设计在更广测试覆盖下的稳定性;
- CoreMark short 与 strict: 评估性能结果并验证长时间运行稳定性;

- Vivado impl50 与 FPGA-like probe: 评估 FPGA 原型可实现性和路径一致性。

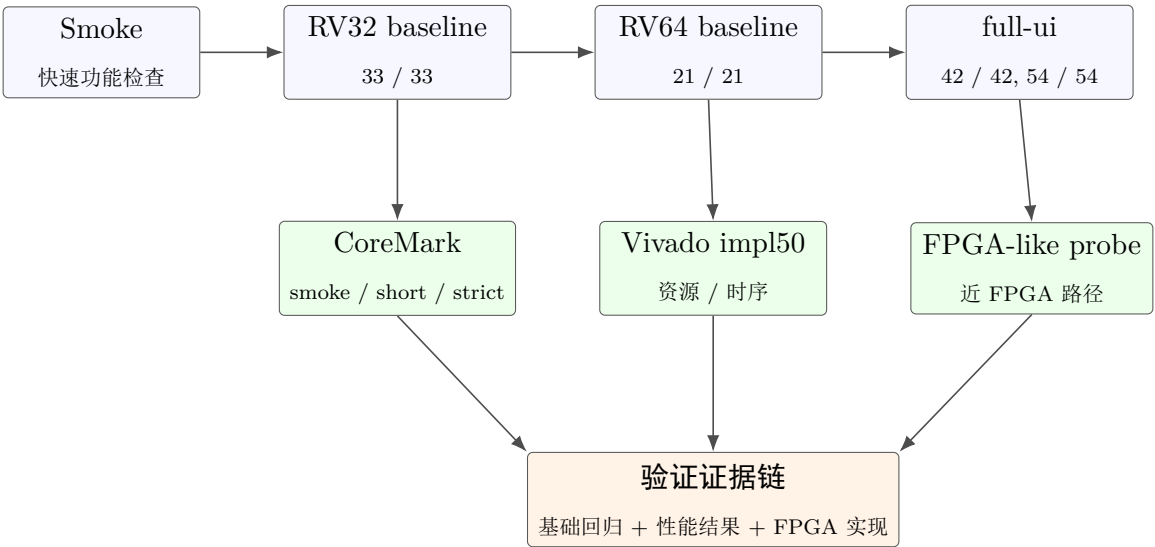


图 3: 验证闭环示意图

3.2 功能验证结果

基础回归验证结果如下：

表 4: 基础回归验证结果

验证项	结果	说明
rv32 baseline	33/33	RV32 基本测试集合全部通过
rv64 baseline	21/21	RV64 基本测试集合全部通过
Smoke 回归	PASS	关键脚本与最小 SoC 软件路径运行正常

为进一步说明设计在更广测试覆盖下的稳定性，还完成了扩展 full-ui 验证：

表 5: 补充用户态验证结果

验证项	结果	说明
rv32 full-ui	42/42	覆盖更广的 RV32 用户态测试集合
rv64 full-ui	54/54	覆盖更广的 RV64 用户态测试集合

3.3 CoreMark 性能结果

CoreMark 采用 “short + strict” 两条路径记录性能：

表 6: CoreMark 结果汇总

路径	Score	周期数	说明
CoreMark short	0.912472	11014885	快速性能评估
	CoreMark/MHz		结果
CoreMark strict >=10s	0.912465	1095991523	长时间运行
	CoreMark/MHz		结果, 满足
			EEMBC 10s 条
			件
FPGA-like probe	7.728811	156442	用于观察近
	CoreMark/MHz		FPGA 路径下
			的相对效率

short 与 strict 两条路径的 Score 保持一致, 说明该设计在长时间运行下没有出现明显性能漂移, 也支撑了本文对性能稳定性的说明。

3.4 FPGA 实现结果

为了支撑 FPGA 原型评估, 项目完成了 Vivado impl50 结果收集:

表 7: Vivado impl50 结果

指标	结果	说明
Slice LUTs	2556	综合后逻辑资源占用
Slice Registers	2170	综合后寄存器占用
Block RAM Tile	4	片上存储资源占用
DSP	0	未依赖 DSP 资源
WNS	+5.822ns	50MHz 目标下保持正裕量
WHS	+0.057ns	保持正保持时间裕量

3.5 验证结论

从上述验证矩阵可以得出三点结论:

1. 处理器在基础 ISA 正确性、CoreMark 性能和 FPGA 综合实现结果之间已经形成一致的证据链;
2. 扩展覆盖结果说明该设计不仅能够完成基础功能, 也具备进一步拓展测试范围的稳定基础;
3. 现有结果足以说明该方案具备较高的工程完成度, 实体板卡相关结论仍需在后续阶段补充。

4 FPGA 原型实现情况

4.1 原型目标与实现路径

赛题允许采用仿真与原型实现结果作为阶段性支撑，因此项目在初赛阶段将 FPGA 工作重点放在“pre-board 可实现性确认”上，即先完成综合、实现、时序、约束和 bring-up 路径整理，再进入实体板验证。本设计以 Nexys A7-100T 为目标平台，并以 Vivado impl150 流程作为 FPGA 原型实现的统一条件。

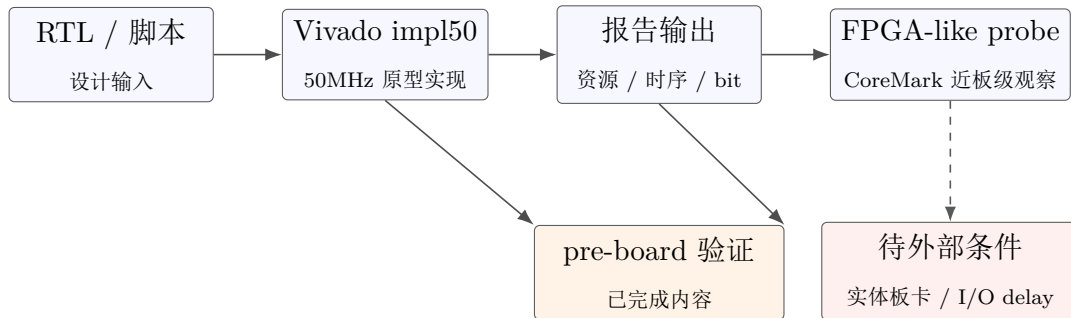


图 4: FPGA pre-board 实现与板级边界示意图

4.2 已完成内容

已经完成的 FPGA pre-board 工作包括：

- Vivado impl150 构建脚本、bitstream 路径和报告目录整理完毕；
- FPGA 顶层、XDC、bring-up checklist 和相关 README 已统一到同一实现配置；
- IMEM_OUTPUT_REG = 0 与 DMEM_OUTPUT_REG = 0 的原型路径配置已固定；
- FPGA-like probe 已用于辅助观察原型路径下的运行周期与相对效率。

4.3 关键结果

表 8: FPGA pre-board 关键结果

项目	结果	说明
目标平台	Nexys A7-100T	原型验证目标板卡
实现条件	impl50	50MHz 目标频率下的综合实现流程
资源结果	2556 LUT / 2170 FF / 4 BRAM / 0 DSP	综合实现后的资源统计
时序结果	WNS = +5.822ns, WHS = +0.057ns	pre-board 约束下保持正时序裕量
原型观测结果	156442 cycles, 7.728811 CoreMark/MHz	来自 FPGA-like probe

4.4 待验证事项

本文给出的 FPGA 结果属于 pre-board 阶段结论，主要用于说明设计具备原型实现能力。尚未完成的内容包括 UART/LED 实体连线验证、boot banner 现场观测以及最终板级 I/O 约束确认。这些工作依赖实体板卡与现场条件，因此被保留为后续阶段任务。

4.5 小结

从现有结果看，YH_rv_cpu 已经完成 FPGA 原型实现的关键前置工作，具备继续进入实体板 bring-up 的工程基础。对于初赛阶段而言，这说明该方案不仅完成了 RTL 设计，也已经具备面向原型实现的明确落地路径。

5 结论

5.1 综合结论

YH_rv_cpu 已经形成较完整的初赛工程形态：架构层面采用边界清晰的五级顺序流水，验证层面建立了从 smoke、baseline、扩展覆盖到 CoreMark、impl50 和 FPGA-like probe 的证据链，原型层面完成了可复现的 FPGA pre-board 适配。综合来看，本设计在初赛阶段具备三项较明确的特点：

- 方案结构清晰，能够较好地说明设计原理与工程取舍；
- 验证矩阵完整，实验结果具备较强的可复核性；
- 资源与时序结果能够支撑后续 FPGA 原型落地，而不是停留在概念设计阶段。

5.2 待继续验证事项

本文也对仍待继续验证的事项作出说明：

- 本文重点说明以 RV32I + Zicsr 为核心的软件运行路径和最小 SoC 实现；
- 扩展 full-ui 与 RV64 验证结果用于补充说明设计覆盖范围和结构稳定性；
- FPGA 结果属于 pre-board 阶段结论，实体板卡验证仍需在后阶段完成。

5.3 结语

总体而言，YH_rv_cpu 的价值不在于追求极端复杂结构，而在于以清晰的设计边界、完整的验证矩阵和可复现的实现结果，建立了一套具备工程完整性的处理器方案。

A 关键结果汇总

表 9: 关键结果汇总

项目	结果	说明
RV32 baseline	33/33	RV32 基本测试集合
RV64 baseline	21/21	RV64 基本测试集合
RV32 full-ui	42/42	扩展 RV32 用户态覆盖
RV64 full-ui	54/54	扩展 RV64 用户态覆盖
CoreMark short	0.912472	周期数 11014885
	CoreMark/MHz	
CoreMark strict	0.912465	周期数 1095991523, 满足 EEMBC
	CoreMark/MHz	10s 条件
FPGA-like probe	7.728811	周期数 156442
	CoreMark/MHz	
Vivado impl50	2556 LUT / 2170 FF	综合实现结果
	/ 4 BRAM / 0 DSP	
时序结果	WNS = +5.822ns,	50MHz 目标下保持正裕量
	WHS = +0.057ns	

本文引用的详细日志、实现报告和回归摘要均保留在项目工程中，用于后续复核与继续演进。